

Hikaya

AI-Powered Platform for Preserving Palestinian Memory and Heritage

Sprint 2: MVP Integration & First Working Prototype

Team: Beta
PM / Issuer: Aws Hammad
Date of Issue: July 26, 2026

Strict Deadline: August 2, 2026 – Before 11:59 PM

1 Document Purpose

This document defines the official Sprint 2 action plan for Team Beta. After completing Sprint 1, the project now moves from initial setup and documentation into technical integration and prototype development.

Sprint 2 focuses on connecting the main components of Hikaya together: backend, database, frontend, AI module, and content files. The goal is to produce the first usable MVP flow where a user can view selected Palestinian locations, open a location page, read related content, generate a simplified AI story, and submit a basic contribution.

Sprint Objective: Build the first integrated MVP prototype of Hikaya by connecting the backend, database, frontend, AI workflow, and content files into one usable system.

2 Sprint Summary

- **Sprint Name:** Sprint 2 – MVP Integration & First Working Prototype
- **Project:** Hikaya – AI-Powered Platform for Preserving Palestinian Memory and Heritage
- **Team:** Beta
- **Project Manager:** Aws Hammad
- **Start Date:** July 26, 2026
- **Deadline:** August 2, 2026, before 11:59 PM

- **Main Goal:** Create the first integrated MVP flow using the work completed in Sprint 1.

3 Sprint 1 Completion Context

Sprint 1 prepared the technical and organizational foundation for the project. The team completed the initial GitHub structure, backend and database planning, frontend setup direction, AI workflow documentation, AI-backend contract, and content drafts for the first MVP locations.

Sprint 2 will build directly on these deliverables and will focus on implementation, connection, and early testing.

Sprint 2 builds on the following completed foundations:

- GitHub repository structure
- Backend project skeleton
- Database schema draft
- Frontend project setup
- AI narrative generation workflow
- AI-backend JSON contract
- Content drafts for Old City of Jerusalem, Bethlehem, and Sebastia

4 Backend & Database Team

Assigned Members

- Nouredin Etkaidek
- Doaa Hatu

Required Deliverables

- **Task 1: Database Implementation & Seed Data**
 - Convert the database schema draft into an actual PostgreSQL implementation.
 - Create the required database tables for the MVP:
 - Locations
 - Stories
 - Users

- Media metadata
 - Content Categories
 - Contributions
- Add initial seed data for the three MVP locations:
 - Old City of Jerusalem
 - Bethlehem
 - Sebastia
- Seed data should include:
 - Location name
 - City or area
 - Short description
 - Latitude and longitude if available
 - Category
 - Related story or content reference
- Push the database implementation files to the **database/** folder.

● Task 2: Working Backend APIs

- Continue the backend implementation using the selected backend framework.
- Implement the following working endpoints:
 - GET `/locations`
 - GET `/locations/{id}`
 - GET `/stories/{location_id}`
 - POST `/contributions`
 - POST `/ai/generate-story`
- The backend should return real or seeded data instead of only placeholder responses.
- The contribution endpoint should accept basic user-submitted story data and store it or prepare it for review.
- Push the updated backend code to the **backend/** folder.

● Task 3: Backend README Update

- Update the backend documentation.
- The **backend/README.md** file must include:
 - Framework used
 - Installation steps
 - Run command

- List of available endpoints
- Example request and response

5 Frontend Team

Assigned Member

- Amra Sawalha

Required Deliverables

- **Task 1: Build Main MVP Pages**

- Implement the first usable version of the main MVP pages.
- The frontend must include:
 - Home page
 - Interactive map page or map placeholder
 - Location details page
 - Story narrative section
 - User contribution form
- The design should follow the Sprint 1 wireframe direction.
- The interface should be clean, simple, and suitable for a cultural heritage platform.
- Push the updated frontend code to the `frontend/` folder.

- **Task 2: Connect Frontend With Backend APIs**

- Connect the frontend with the backend endpoints prepared by the backend team.
- The frontend should be able to:
 - Fetch the list of locations
 - Display the three MVP locations
 - Open a selected location
 - Display the location details and related story
 - Send a user contribution using the contribution form
- If backend integration is not fully ready, use temporary mock data with the same final data structure.
- Replace mock data with backend responses once the APIs are ready.

- **Task 3: Frontend README Update**

- Update the `frontend/README.md` file.
- Include:
 - Framework used
 - Installation steps
 - Run command
 - Main pages
 - Current integration status

6 AI Smart Features Team

Assigned Members

- Hussein Jaradat
- Arwa Al-Hajahja

Required Deliverables

- **Task 1: Implement First AI Story Generation Function**
 - Convert the Sprint 1 AI workflow into a working function or script.
 - The function should receive reviewed historical content and return a simplified story.
 - The function should support at least two modes:
 - Student mode
 - Tourist mode
 - If possible, also support:
 - Child-friendly mode
 - Short summary mode
 - The function must follow the main AI safety rule:
 - Do not invent unsupported historical facts.
 - Use only the provided reviewed content.
 - Return an uncertainty note if the provided content is not enough.
 - Push the implementation to the `ai/` folder.
- **Task 2: Test AI Using MVP Content**
 - Test the AI function using the content files prepared for:
 - Old City of Jerusalem
 - Bethlehem

- Sebastia
- Generate at least one sample story for each location.
- Store the generated samples inside the `ai/examples/` folder.
- Each sample should mention:
 - Input location
 - Target audience
 - Generated story
 - Notes about limitations or missing information
- **Task 3: AI Endpoint Integration Support**
 - Coordinate with the backend team to support the POST `/ai/generate-story` endpoint.
 - Confirm the final request and response JSON format.
 - Provide a simple example that backend developers can use for testing.
 - Update the AI-backend contract if any changes are needed.

7 Content Documentation Team

Assigned Member

- Tamara Hamdan

Required Deliverables

- **Task 1: Prepare AI-Ready Summaries**
 - Create shorter AI-ready summaries from the full content files.
 - Prepare one summary file for each MVP location:
 - `old-city-jerusalem-summary.md`
 - `bethlehem-summary.md`
 - `sebastia-summary.md`
 - Each summary should include:
 - Short overview
 - Key historical timeline
 - Cultural and religious importance
 - Main landmarks
 - Important facts
 - Source notes

- Upload the summary files to the `content/ai-ready/` folder.
- **Task 2: Improve References and Media Links**
 - Review the existing content files and improve the source section.
 - Add direct links to reliable references when possible.
 - Add image links or media references for each location if available.
 - Clearly separate:
 - Historical facts
 - Religious traditions
 - Popular stories or oral heritage
 - This will help the AI team avoid treating traditions as confirmed historical facts.
- **Task 3: Content README Update**
 - Update the `content/README.md` file.
 - Explain the difference between:
 - Full content files
 - AI-ready summary files
 - Media links
 - Source notes

8 Project Management Tasks

Assigned Member

- Aws Hammad

Required Deliverables

- **Task 1: GitHub Review and Pull Request Management**
 - Review all Sprint 2 branches before merging into `main`.
 - Make sure each sub-team uses clear branch names and descriptive commits.
 - Confirm that all files are uploaded to the correct folders.
 - Track unfinished or blocked tasks.
- **Task 2: Integration Coordination**
 - Coordinate between frontend and backend teams.
 - Coordinate between backend and AI teams.

- Make sure the API formats are clear and agreed on.
- Arrange short check-ins when integration problems appear.
- **Task 3: Sprint 2 Completion Report**
 - Prepare the Sprint 2 completion report after the deadline.
 - Summarize completed work, GitHub updates, challenges, and next steps.
 - Include the GitHub repository link and mention the main completed deliverables.

9 Sprint 2 Integration Requirements

By the end of Sprint 2, the team should have a simple integrated user flow.

Minimum Integrated Flow:

- User opens the frontend.
- User sees the three MVP locations.
- User selects one location.
- Frontend displays location details and related content.
- User requests an AI-generated story.
- Backend sends the selected content to the AI module.
- AI returns a simplified narrative.
- Frontend displays the generated story.
- User submits a contribution through the contribution form.

If full integration is not completed, the team must still provide:

- Working frontend with mock data.
- Working backend endpoints tested independently.
- Working AI function tested independently.
- Clear documentation showing how the components will connect.

10 General GitHub Rules

All team members must continue following the GitHub workflow during Sprint 2.

- **No Direct Work on Main**
 - No one should push directly to the `main` branch.

- Each sub-team must work on a separate branch.
- Pull requests should be reviewed before merging.
- **Required Branch Names**
 - Backend branch: `backend-api-integration`
 - Database branch: `database-seed-data`
 - Frontend branch: `frontend-mvp-pages`
 - AI branch: `ai-story-generation`
 - Content branch: `content-ai-ready-summaries`
 - Documentation branch: `docs-sprint-2`
- **Commit Rules**
 - Use regular and descriptive commits.
 - Each commit should describe a real change.
 - Avoid vague commit messages such as `update`, `fix`, or `changes`.
 - Examples of good commit messages:
 - `Implement locations API endpoint`
 - `Add seed data for MVP locations`
 - `Create location details page`
 - `Add AI story generation function`
 - `Prepare AI-ready Jerusalem summary`
- **Daily Progress Updates**
 - Each active member should send a short progress update before the end of the day.
 - The update should include:
 - What was completed
 - What will be done next
 - Any blocker
 - GitHub commit link or screenshot if available

11 Sprint 2 Final Deliverables Checklist

By August 2, 2026 before 11:59 PM, the following deliverables must be available on GitHub:

- PostgreSQL database implementation or updated schema inside `database/`

- Seed data for Old City of Jerusalem, Bethlehem, and Sebastia
- Working backend endpoints for locations, stories, contributions, and AI story generation
- Updated backend README with run instructions and endpoint documentation
- First usable frontend pages inside `frontend/`
- Frontend connected to backend APIs or using matching mock data
- User contribution form implemented on the frontend
- AI story generation function or script inside `ai/`
- Sample generated stories for the three MVP locations
- Updated AI-backend JSON contract if needed
- AI-ready content summaries inside `content/ai-ready/`
- Improved source and media references inside the content files
- Regular descriptive commits from each active sub-team
- Pull requests reviewed before merging into `main`

12 Definition of Done

A Sprint 2 task is considered complete only if:

- The assigned deliverable is pushed to GitHub.
- The file or code is placed in the correct project folder.
- The work has a clear and descriptive commit message.
- The task has enough documentation for another team member to understand it.
- Any required integration format is documented.
- The PM has reviewed the task or pull request.

Important Rule: Work that is not pushed to GitHub will not be counted as completed sprint work.

13 Closing Note

Sprint 2 is a critical step for Hikaya because it moves the project from separate setup tasks into a connected MVP prototype. The focus is now on integration, real data, working APIs, usable frontend pages, and the first AI-generated storytelling output.

All team members are expected to complete their assigned tasks before the deadline and keep their progress visible on GitHub. The goal is to finish this sprint with a clear prototype foundation that can be tested, improved, and presented in the following phase.

Issued by: Aws Hammad

Role: Project Manager, Team Beta

Project: Hikaya – AI-Powered Platform for Preserving Palestinian Memory and Heritage